

```
    ;return total
}
```

همینه، تموم. 👍 ۲-۳ خط بدنه حلقه رو نمی‌بریم توی متد جدا. وسواس رو بذار کنار.

معيار تصميم‌گيري 🛠️

بدنه حلقه	تصميم
۱-۵ خط	✅ توی همون متد بمونه
۵-۱۵ خط	😐 فکر کن، اگه منطق مستقلي داره Extract کن
۱۵+ خط	❌ حتماً Extract کن

پس بيا همينجا Long Method رو دفن کنیم و بریم Large Class! 🍌

لینکش رو بفرست 😊

<https://refactoring.guru/smells/large-class>

.Link reading is unavailable in Expert Mode. Please use Instant Mode

Large Class (کلاس بزرگ) 🐘

تعريف ساده

کلاسی که بیش از حد کار انجام میده و پر شده از فیلدها، متدها و مسئولیت‌های مختلف. به این کلاس‌ها God Object (شیء خداگونه) هم گفته میشه.

چرا Large Class بوی بد محسوب میشه؟ 🧠

مشکل	توضیح
نقض Single Responsibility	کلاس چندین دلیل برای تغییر داره
خوانایی کم	پیدا کردن یه متد بین ۵۰ تا متد دیگه سخت میشه
کمک به Duplicate Code	منطق‌های مشابه توی بخش‌های مختلف کلاس پراکنده میشن
تست سخت	تست کردن کلاسی با کلی وابستگی و فیلد، کابوسه
Merge Conflict	توی تیم، همه مجبورن همین فایل رو تغییر بدن → conflict زیاد

نشانه‌های Large Class

نشانه	توضیح
تعداد زیاد فیلدها	+۱۰ فیلد که به گروه‌های مختلف تعلق دارن
تعداد زیاد متدها	+۱۵ متد که کارهای مختلف انجام میدن
کامنت‌های جداکننده	region Validation# یا --- User Management --- //
کلاس با چندین وجه	یه کلاس Employee که هم حقوق حساب می‌کنه، هم مرخصی، هم گزارش
نام کلاس خیلی کلی	Processor , Service , Helper , Manager

تکنیک‌های درمان (پیش‌نمایش)

تکنیک	کاربرد
Extract Class	فیلدها و متدهای مرتبط رو ببر به کلاس جدید

بخشی از رفتار رو ببر به زیرکلاس	Extract Subclass
رابط‌های جدا برای هر گروه از متدها بساز	Extract Interface

مثال ساده (قبل از ریفکتور)

csharp

```
public class Employee
{
    // اطلاعات شخصی
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Phone { get; set; }
    public string Address { get; set; }

    // اطلاعات شغلی
    public string Department { get; set; }
    public decimal Salary { get; set; }
    public DateTime HireDate { get; set; }

    // اطلاعات مالی
    public string BankAccount { get; set; }
    public string TaxCode { get; set; }

    // متدهای مختلف با مسئولیت‌های متفاوت
    { /* ... */ } ()public decimal CalculateSalary
    { /* ... */ } public void UpdateAddress(string newAddress)
    { /* ... */ } ()public void GeneratePaySlip
    { /* ... */ } ()public void CalculateTax
    { /* ... */ } ()public bool IsEligibleForPromotion
    { /* ... */ } public void SendEmail(string message)
}
```

اینجا Employee حداقل ۳ تا شخصیت داره: پرسنلی، شغلی، مالی.

Extract Class بعد از

csharp

```

        public class Employee
        {
            public PersonalInfo Personal { get; set; }
            public JobInfo Job { get; set; }
            public FinancialInfo Financial { get; set; }
        }

        public class PersonalInfo
        {
            public string FirstName { get; set; }
            public string LastName { get; set; }
            public string Phone { get; set; }
            public string Address { get; set; }
        }

        public class JobInfo
        {
            public string Department { get; set; }
            public decimal Salary { get; set; }
            public DateTime HireDate { get; set; }
        }

        { /* ... */ } ()public bool IsEligibleForPromotion
        {

            public class FinancialInfo
            {

                public string BankAccount { get; set; }
                public string TaxCode { get; set; }

            }

            { /* ... */ } ()public void GeneratePaySlip
            { /* ... */ } ()public void CalculateTax
            {

```

مزیت	توضیح
هر کلاس یه کار داره	FinancialInfo فقط کار مالی
تغییرات محدود	تغییر در بخش مالی فقط FinancialInfo رو عوض می‌کنه
تست آسون‌تر	FinancialInfo رو جدا تست می‌کنی
خوانایی بالا	می‌دونی هر چیزی کجاست

تمرین

این OrderManager رو با **Extract Class** درمان کن:

csharp

```
public class OrderManager
{
    // اطلاعات مشتری
    public string CustomerName { get; set; }
    public string CustomerEmail { get; set; }
    public string CustomerPhone { get; set; }
    public string CustomerAddress { get; set; }

    // اطلاعات سفارش
    public List<OrderItem> Items { get; set; } = new
    public DateTime OrderDate { get; set; }
    public string Status { get; set; }

    // اطلاعات پرداخت
    public string PaymentMethod { get; set; }
    public string CardNumber { get; set; }
    public decimal Amount { get; set; }

    // اطلاعات ارسال
    public string ShippingAddress { get; set; }
    public string TrackingCode { get; set; }
    public DateTime? ShippedDate { get; set; }

    // متدها
```

```

{ /* ... */ } public void AddItem(string name, decimal price, int quantity)
{ /* ... */ } public void RemoveItem(string name)
    ()public decimal CalculateTotal
    }

    ;decimal total = 0
    foreach (var item in Items)
;total += item.Price * item.Quantity
    ;return total
    {

    { /* ... */ } ()public void ProcessPayment
    { /* ... */ } ()public void RefundPayment

    { /* ... */ } ()public void ShipOrder
{ /* ... */ } public void UpdateTracking(string code)

    { /* ... */ } ()public void SendConfirmationEmail
    { /* ... */ } ()public void SendShippingNotification

{ /* ... */ } public void UpdateCustomerInfo(string name, string email)
    {

```

راهنمایی: می‌تونی به این کلاس‌ها بشکنیش:

- CustomerInfo - اطلاعات مشتری + UpdateCustomerInfo
- OrderDetails - آیتم‌ها، تاریخ، وضعیت + AddItem , RemoveItem , CalculateTotal
- PaymentInfo - پرداخت + ProcessPayment , RefundPayment
- ShippingInfo - ارسال + ShipOrder , UpdateTracking

بفرست تا بررسی کنیم! 🍷

این موارد درمان هست
این مورد آخری هم جز اون هست که نگفتیش

Treatment

When a class is wearing too many (functional) hats, think about splitting
:it up

Extract Class helps if part of the behavior of the large class can be spun
.off into a separate component

Extract Subclass helps if part of the behavior of the large class can be implemented in different ways or is used in rare cases

Extract Interface helps if it's necessary to have a list of the operations and behaviors that the client can use

If a large class is responsible for the graphical interface, you may try to move some of its data and behavior to a separate domain object. In doing so, it may be necessary to store copies of some data in two places and keep the data consistent. Duplicate Observed Data offers a way to do this

درسته! ببخشید، Duplicate Observed Data رو جا انداختم. ممنون که یادآوری کردی.

لیست کامل درمان Large Class

#	تکنیک	کاربرد
۱	Extract Class	بخشی از رفتار رو ببر به کلاس جدید
۲	Extract Subclass	بخشی از رفتار پیاده‌سازی‌های مختلف داره
۳	Extract Interface	نیاز به لیست عملیات‌های مجاز برای client
۴	Duplicate Observed Data	وقتی کلاس بزرگ مسئول UI هم هست

Duplicate Observed Data (خلاصه)

مخصوص وقتی که یه کلاس هم منطق تجاری داره هم منطق رابط کاربری (UI). راه درمان: داده‌های UI رو ببر به یه کلاس جدا (Domain Object) و توی UI یه کپی ازش نگه دار. اینطوری مسئولیت‌ها جدا میشن.

حالا بیا به ترتیب کار کنیم. Extract Class رو با همون تمرین قبلی انجام بده.